

Understanding Anime Success: Predicting Popularity and Genre Influence Using Machine Learning

Mohammad Hasibul Amin

*Dept. of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh*

Fabiha Tarannum Areena

*Dept. of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh*

Zawad Ahsan

*Dept. of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh*

Abstract—The anime industry is in its extreme blooming period. And nowadays, anime releases are not niche anymore. It has reached mainstream media, and it has a huge audience. But the increase makes it extremely difficult to identify which title will achieve high audience ratings even before they air. But the fact is, there is a lot of rich metadata available. As an example, the metadata available on the platform of MyAnimeList is so vast, but still, the information is rarely used for systematic predictive analysis. So this paper mainly targets two main problems. We try to predict anime success using metadata available before a title is even released, and we try to understand how genre combinations influence audience scores. So we applied machine learning to the MyAnimeList dataset [10] of 14,478 anime titles across around 30 attributes. We used a seven-step preprocessing pipeline. There are multi-label genre encoding and engineered episode category features available. We used seven regression models to train and evaluate the data. There are four classical baselines, such as linear regression, decision tree, random forest, and XGBoost. Three neural network architectures, such as shallow, deep, and wide and deep. We assessed using RMSE, MSE, and R-squared on an 80/20 train-test split. From the baseline models, random forest leads with test RMSE 0.4404 and R-squared 0.9115. Among the neural networks, the Wide and Deep neural network leads with test RMSE 0.5557 and R-squared 0.8590. We found drama to be the most influential genre feature, while high-frequency genres such as comedy and action showed limited discriminative power. These findings demonstrate that ensemble and deep learning methods can effectively model anime success from metadata, and we can establish a benchmark for future pre-release prediction systems.

Index Terms—anime, score prediction, genre combination, machine learning, random forest, XGBoost, neural network, MyAnimeList, regression

I. INTRODUCTION

The global anime industry produces hundreds of new titles every year, yet predicting which series will achieve high audience ratings remains a non-trivial challenge. Platforms such as MyAnimeList (MAL) aggregate rich metadata per title

including genre, episode count, source material, age rating, and user scores. Despite this, it is rarely exploited for systematic predictive analysis. Studios and streaming services would benefit from tools that estimate a title's reception before it airs.

This paper addresses two questions: (i) **predicting anime success before release** using metadata knowable at announcement time, and (ii) **genre-combination impact on ratings** beyond individual genre effects. Contributions include a seven-step preprocessing pipeline, a comparative evaluation of seven regression models (four classical and three neural network architectures), and a feature-level genre analysis.

II. RELATED WORK

Content-based and collaborative filtering dominate anime data research. Reynaldi and Istiono [1] achieve 74.23% user satisfaction with a scraped-metadata content-based system. Chen [8] advocates hybrid frameworks combining collaborative filtering with boosting algorithms to address catalogue overabundance. Zhang [2] introduces a self-evaluating collaborative filtering algorithm showing that recommending ten titles at a time yields the most stable predictions. Meng [4] reports that collaborative methods outperform content-based ones on large rating datasets. None of these works address cold-start score prediction for new titles before any ratings exist.

Deep learning approaches have also been explored. Soni et al. [3] combine deep autoencoders and spectral clustering in RikoNet, achieving RMSE 0.591 on MovieLens. Javaji and Sarode [5] treat recommendation as link prediction using GraphSAGE and BERT. Saito et al. [9] find that combining metadata-based and synopsis-based knowledge graphs yields the strongest GNN-based performance. Kang et al. [7] show that removing all multi-modal features drops model performance by 9%. Sosa et al. [6] model a 325,000-user bipartite

network using an ERGM, finding positive coefficients for Fantasy and Supernatural clusters, motivating our feature-level genre-combination analysis.

III. DATASET DESCRIPTION

The **MyAnimeList Dataset** [10] contains 14,478 anime titles across 30 columns. Key attributes are: *type* (TV, Movie, OVA, etc.), *episodes*, *status*, *source*, *genre* (comma-separated multi-label), *duration*, *rating* (G through Rx), *score* (1–10, **target**), and engagement metrics *members*, *favorites*, *scored_by*, *rank*, *popularity*.

A. Exploratory Data Analysis

Fig. 1 shows that scores follow a bell-shaped distribution centred at 6.5–7.5, with very few titles scoring below 4 or above 9. This relatively narrow target range makes exact score prediction non-trivial, especially for linear models. TV series are the most frequent type, followed by OVAs and Movies.

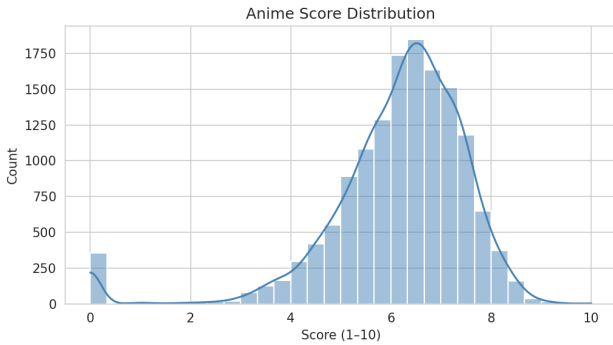


Fig. 1. Anime score distribution. Most titles score between 6 and 8, indicating a bell-shaped distribution centred around 6.5–7.5.

Fig. 2 shows the ten most frequent genres. Comedy leads with over 4,500 entries, followed by Action and Fantasy. Drama, while ranked fifth in frequency, proves to be the most predictively useful genre feature. High-frequency genres tend to be ubiquitous across both high-rated and low-rated titles, reducing their signal strength. Columns *background* (>80% missing) and *licensor* ($\approx 70\%$ missing) are dropped before modelling.

IV. METHODOLOGY

A. Workflow Overview

Fig. 3 illustrates the end-to-end pipeline from raw data to final evaluation.

B. Data Preprocessing

The preprocessing pipeline consists of seven ordered steps, designed to prevent data leakage and any type of errors throughout the workflow.

In the first step, before any imputation is performed, all identifier, free-text, and date columns are removed. They include fields such as anime ID, all title variants, synopsis, background, studio, air dates, broadcast, premiere, related

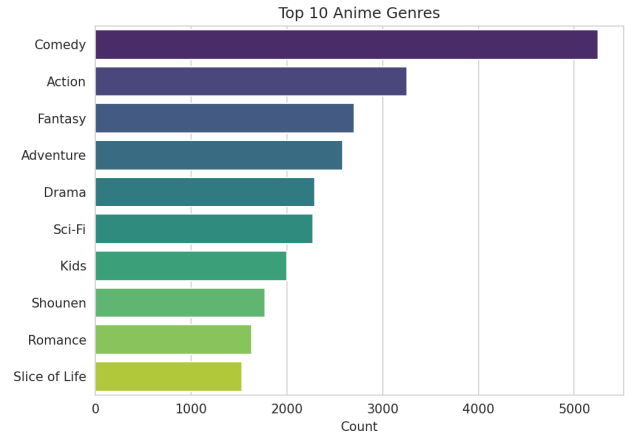


Fig. 2. Top 10 genres by frequency. Comedy, Action, and Fantasy dominate. High-frequency genres show limited discriminative power for score prediction.

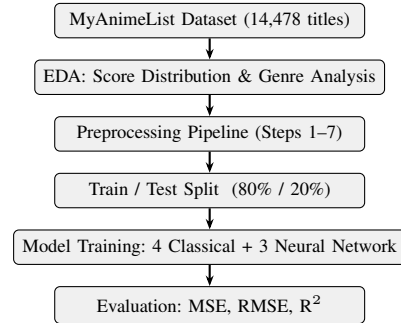


Fig. 3. End-to-end project workflow from raw dataset to model evaluation.

producer, licensor, and airing status. Removing these columns before imputation prevents the mode-fill step from producing incorrect values in columns that are mostly empty.

In the second step, two new features are engineered from existing raw columns. The episode count is divided into episode categories: 12 or fewer, 13 to 24, 25 to 50, 51 to 200, and over 200. The duration column contains values as a string. The general form of these strings is parsed to replace the string with the numeric value in the column *duration_min*. This is to avoid the error of type during scaling. The missing values are filled with the median of the column for numeric variables, and the mode of the column for categorical variables.

In the third step, missing values are filled using the column median for all numerical features, which is robust to outliers, and the column mode for all categorical features.

In the fourth step, since each anime is assigned to a variable number of genres, genre encoding is approached as a multi-label problem. The usual method of one-hot encoding is not applicable, so the ten most common genres are identified and assigned their own separate binary columns.

In the fifth step, the remaining categorical columns like *type*, *source*, *rating*, *status*, and *episode_category* are converted into numbers using one-hot encoding. The *drop_first=True* setting is used

to avoid redundancy in the data. A quick check is run to make sure no text columns are left before scaling starts. In the sixth step, the data is split into two parts. 80% is used for training which is 11,582 samples and 20% is used for testing which is 2,896 samples. A fixed random state of 42 is set to keep results consistent.

In the sixth step, the dataset is split into two parts: 80% for training (11,582 samples) and 20% for testing (2,896 samples) with `random_state=42` for reproducibility.

In the seventh step, `StandardScaler` is fitted only on the training set and then applied to both sets, preventing leakage. The target score is never scaled. The scaled matrix is used for Linear Regression and all neural networks; tree-based models use the unscaled matrix.

C. Models

Linear Regression assumes a linear relationship between features and target score, trained on the scaled matrix. Its coefficients indicate each feature’s contribution.

Decision Tree Regressor applies recursive binary splits with `max_depth=10` to prevent memorisation. Scale-invariant, trained on the unscaled matrix.

Random Forest Regressor aggregates 100 decision trees, each on a bootstrap sample with random feature subsets, reducing the high variance of individual trees.

XGBoost Regressor builds trees sequentially, each correcting prior errors. Learning rate 0.1, 200 estimators, max depth 6.

Shallow Neural Network uses three hidden layers (128, 64, 32 units) with ReLU, BatchNormalization, and Dropout. Adam at 10^{-3} , max 100 epochs, EarlyStopping patience 10. Stopped at epoch 36.

Deep Neural Network uses five hidden layers (256, 128, 64, 32, 16) with L2 regularisation on the two widest layers and higher Dropout (0.4 and 0.3). ReduceLROnPlateau halves learning rate after 7 stalling epochs. Stopped at epoch 60.

Wide and Deep Neural Network uses a dual-path design. The wide path passes all 48 raw features directly to a merge layer to capture linear relationships. The deep path applies three dense layers (256, 128, 64) before merging. Concatenated outputs pass through Dense(32) to the output. ReduceLROnPlateau applied. Stopped at epoch 46.

D. Evaluation Metrics

Mean Squared Error (MSE) [11] measures the average squared deviation between predicted and actual scores:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1)$$

Root Mean Squared Error (RMSE) [11] returns the error to the original score scale (1–10), making it directly interpretable:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2)$$

R-squared [11] measures the proportion of score variance explained by the model, where 1.0 is a perfect fit:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3)$$

All three metrics are reported on both training and test sets. A large gap between train and test values identifies overfitting.

V. RESULTS AND EVALUATION

A. Classical Model Performance

Table I presents the complete performance comparison of all seven models across MSE, RMSE, and R^2 . **Random Forest** achieves the best overall regression performance (test RMSE 0.4404, test MSE 0.1940, R^2 0.9115). Linear Regression shows almost no train/test gap (train RMSE 0.6409 vs. test RMSE 0.6420), confirming no overfitting, but its linear assumption fails to capture the non-linear interactions in the data. Decision Tree (test RMSE 0.4805) outperforms Linear Regression while `max_depth=10` prevents severe memorisation. XGBoost is competitive with Random Forest on the ranking task (R^2 0.8797, test MSE 0.2634), though it does not exceed Random Forest’s test MSE.

TABLE I
ALL 7 MODELS: MSE, RMSE, AND R^2 (SORTED BY TEST RMSE)

Model	Type	Tr.MSE	Te.MSE	Tr.RMSE	Te.RMSE	Tr. R^2	Te. R^2
Random Forest	ML	0.0303	0.1940	0.1740	0.4404	0.9858	0.9115
Decision Tree	ML	0.2053	0.2309	0.4531	0.4805	0.9037	0.8946
XGBoost	ML	0.0628	0.2634	0.2505	0.5132	0.9706	0.8797
Wide & Deep NN	NN	0.2228	0.3088	0.4720	0.5557	0.8954	0.8590
Deep NN	NN	0.3517	0.3605	0.5930	0.6004	0.8349	0.8354
Shallow NN	NN	0.3543	0.3622	0.5952	0.6018	0.8337	0.8347
Linear Reg.	ML	0.4108	0.4122	0.6409	0.6420	0.8073	0.8118

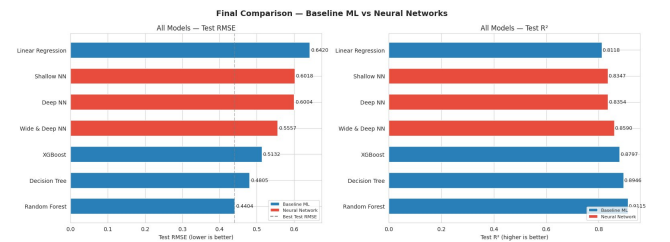


Fig. 4. Test RMSE (left) and Test R^2 (right) for all 7 models. Blue bars are classical ML; red bars are Neural Networks. Random Forest achieves the best test RMSE (0.4404) and test R^2 (0.9115).

B. Neural Network Performance

Among the three neural network architectures, Wide and Deep NN achieves the best regression performance: test RMSE 0.5557, test MSE 0.3088, and R^2 0.8590. Shallow NN (test RMSE 0.6018) and Deep NN (test RMSE 0.6004) perform comparably, with only marginal differences attributable to the

additional depth in the Deep NN. Fig. 5 shows the RMSE and R^2 comparison across all three architectures.

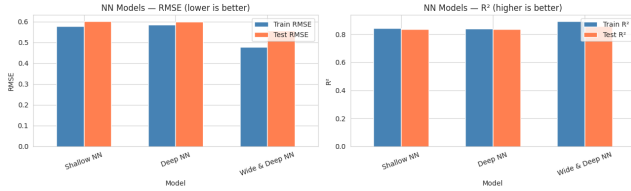


Fig. 5. Train/Test RMSE (left) and R^2 (right) for the three neural network models. Wide & Deep NN achieves the best test RMSE (0.5557) and test R^2 (0.8590).

Fig. 6 shows the overlaid validation MSE loss curves during training. All three networks converge steeply in the first 10 epochs and then stabilise. Wide and Deep NN converges to the lowest validation loss of the three, consistent with its superior test MSE. The step-wise drops visible in the Deep NN and Wide and Deep NN curves indicate the ReduceLROnPlateau callback halving the learning rate when progress stalls.

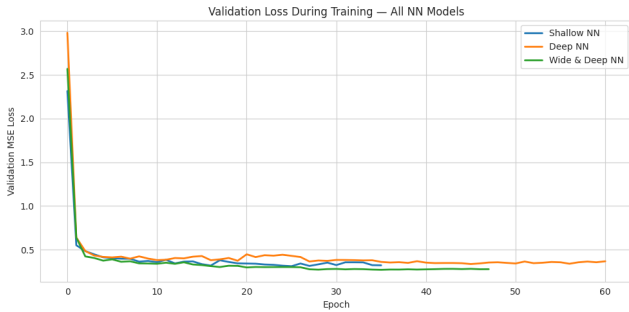


Fig. 6. Overlaid validation MSE loss curves during training for all three neural networks. Wide & Deep NN converges to the lowest validation loss.

C. Actual vs. Predicted and Residual Analysis

Fig. 7 shows scatter plots of actual vs. predicted scores (top row) and residual histograms (bottom row) for the three neural network models. All three scatter plots cluster tightly around the perfect-fit diagonal, confirming that none of the models suffer from systematic bias across the score range. The residual histograms are centred close to zero with a low standard deviation (around 0.55 for all three models). This tells us that the prediction errors are fairly symmetric and follow a normal distribution around the mean. The Wide and Deep Neural Network has the smallest spread of residuals (Std $\approx$ 0.550). This matches the fact that it also achieved the lowest test MSE. Looking at the residual analysis as a whole, the regression models are performing as they should. None of them shows a consistent pattern of predicting too high or too low.

D. Overfitting Analysis

Fig 8 compares the train and test RMSE on the left and R^2 scores on the right across all seven models. This helps us

see how well each model generalises to new data. Random Forest has the largest gap between its training and test results. It scored a train RMSE of 0.1740 but a test RMSE of 0.4404. That is a difference of 0.2664. This tends to happen with ensemble models because they pick up very specific patterns from the training data that do not always carry over to new data. XGBoost has a similar issue, though its gap is a bit smaller at 0.2627. In contrast, neural networks with Dropout and BatchNormalization exhibit tight train/test agreement: Shallow NN gap 0.0066, Deep NN 0.0074, and Wide and Deep NN 0.0837. This confirms that regularisation is more effective at controlling overfitting in neural models than depth alone.

E. Feature Importance

Platform engagement metrics (`rank`, `scored_by`, `popularity`, `members`, `favorites`) are the strongest predictors across all models, as shown in Fig. 9. This result reflects the strong co-occurrence of community engagement and high user scores. Among the ten encoded genre binary features, Drama ranks highest. Common genres such as Comedy and Action do not strongly differentiate high-scoring titles, as their high prevalence dilutes the discriminative signal.

VI. DISCUSSION

A. Predicting Anime Success Before Release

Tree-based ensembles substantially outperform Linear Regression (best test R^2 0.9115 vs. 0.8118), confirming non-linear relationships between metadata and score. Wide and Deep NN (R^2 0.8590) also outperforms the linear baseline, and generalises more reliably due to its narrow train/test MSE gap (0.3088 test vs. 0.2228 train). Random Forest achieves a test MSE of 0.1940, more than halving the test MSE of Linear Regression (0.4122).

A critical caveat is that the strongest predictors, `rank`, `scored_by`, `members`, and `favorites`, are post-release engagement metrics unavailable before a title airs. Including these features improves the reported performance but works against the actual goal of predicting ratings before a show is released. A true pre-release model should only use information that is available at the time of announcement. This includes genre, type, source, episode count, age rating, and duration. Neural networks with Dropout layers are a better fit for this kind of restricted setup. Their small gaps between training and test MSE show that they generalise more consistently compared to ensemble methods.

B. Genre Combination Impact on Ratings

Drama stands out as the most useful genre in the Random Forest feature importances. It gives the model something meaningful to work with. Genres like Comedy and Action are a different story. They show up so often in the dataset that they do not really help the model tell apart the better-rated shows from the rest. Their high frequency is actually what makes them less useful. The current setup treats each genre as

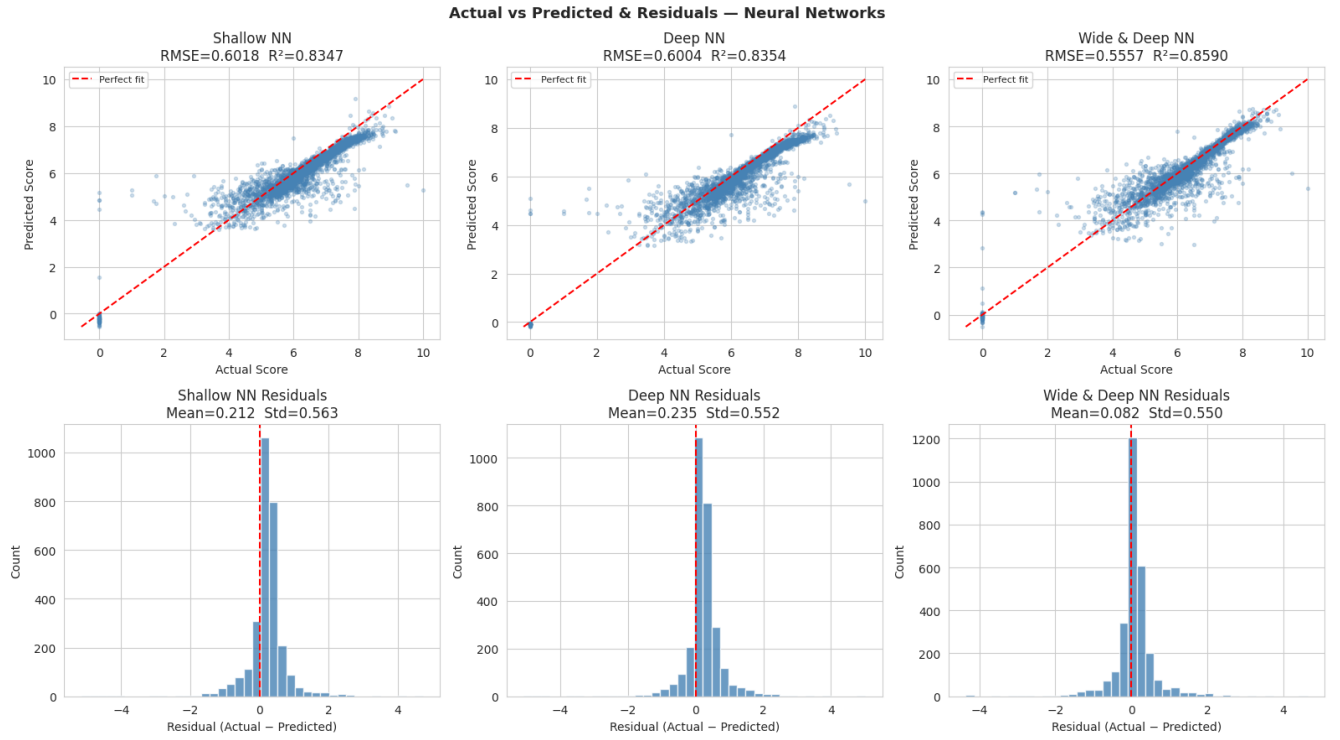


Fig. 7. Actual vs. Predicted scatter plots (top) and residual histograms (bottom) for Shallow NN (RMSE=0.6018, $R^2=0.8347$), Deep NN (RMSE=0.6004, $R^2=0.8354$), and Wide & Deep NN (RMSE=0.5557, $R^2=0.8590$). Residuals are centred near zero across all three models.



Fig. 8. Train vs. Test RMSE (left) and R^2 (right) for all 7 models. Random Forest shows the largest overfitting gap. Neural networks with Dropout achieve more balanced generalisation.

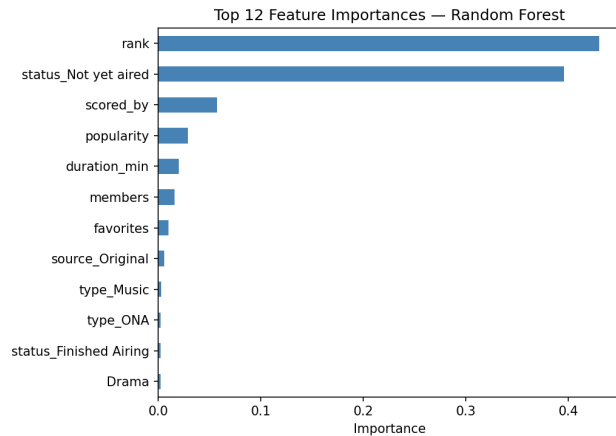


Fig. 9. Top-12 feature importances from Random Forest. Engagement metrics dominate. Among genre features, Drama ranks highest.

a separate binary feature. This only captures the effect of each genre on its own and misses how genres work together. Future work could look at adding genre pair features or using a multi-label learning approach to capture those combined effects. This idea is backed by the findings of Sosa et al. [6], whose work identified positive relationships for Fantasy and Supernatural genre clusters. This suggests that genre interactions do carry useful predictive information worth exploring further.

VII. CONCLUSION

This paper investigated how accurately anime audience success can be predicted from metadata and how genre combinations influence ratings. Using the MyAnimeList dataset [10] of 14,478 titles, a seven-step preprocessing pipeline was applied and seven regression models were evaluated. Random Forest achieved the best classical performance (RMSE 0.4404, MSE 0.1940, R^2 0.9115). Wide and Deep NN led among neural networks (RMSE 0.5557, MSE 0.3088, R^2 0.8590). Neural networks exhibited substantially more balanced train/test behaviour, confirming the benefit of Dropout and Batch Normalization. Drama was found to be the most influential genre feature. Planned future work includes: (i) a restricted pre-release model using only announcement-time metadata; (ii) explicit genre-pair interaction features; (iii) cross-validation for robust performance estimates; and (iv) hyperparameter optimisation for all models.

REFERENCES

- [1] Reynaldi and W. Istiono, "Content-based filtering and web scraping in website for recommended anime," *Asian J. Research in Computer Science*, 2023, doi: 10.9734/ajrcos/2023/v15i2318.
- [2] B. Zhang, "A self-evaluating collaborative filtering recommendation algorithm: A case study of anime recommendations," *Theoretical and Natural Science*, 2024, doi: 10.54254/2753-8818/53/20240151.
- [3] B. Soni, D. Thakuria, N. Nath, N. Das, and B. Boro, "RikoNet: A novel anime recommendation engine," *arXiv:2106.12970*, 2021.
- [4] Z. Meng, "Research on two popular recommendation algorithms for anime," *Applied and Computational Engineering*, 2023, doi: 10.54254/2755-2721/6/20230898.
- [5] S. R. Javaji and K. Sarode, "Hybrid recommendation system using graph neural networks and text embeddings," *arXiv:2310.04878*, 2023.
- [6] J. Sosa, A. Urrego-Lopez, and C. Prieto, "Analysis of bipartite networks in anime series," *arXiv:2411.03333*, 2024.
- [7] Y. Kang, H. Lin, M. Yang, and S. J. Lee, "User-aware multi-modal animation recommendation system," *arXiv:2402.10381*, 2024.
- [8] J. Chen, "The investigation on anime-themed recommendation systems," *Highlights in Science, Engineering and Technology*, vol. 81, pp. 121–131, 2024, doi: 10.54097/36drh331.
- [9] Y. Saito, S. Egami, Y. Sei, Y. Tahara, and A. Ohsuga, "Empirical analysis of knowledge representation for anime recommendation using graph neural networks," *Trans. Japanese Society for AI*, vol. 39, no. 6, 2024, doi: 10.1527/tjsai.39-6_AG24-D.
- [10] Azathoth42, "MyAnimeList dataset," Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/azathoth42/myanimelist>
- [11] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2021.